HDOC Day-58

****

Write a C program that reads the size and elements of a variable-length integer array, arr[], and an integer k such that 0 < k <= size. Pass the array to a function named kth_smallest() using its address. Use pointers to pass the array between the calling function and kth_smallest(). The function kth_smallest() must:

- Arrange the elements of arr[] in ascending order using bubble sort algorithm.
- Find the element that occupies the kth position in the sorted array.
- Display the kth smallest element.
- Consider duplicate elements as separate elements.
- Display -1 if k is invalid.

1. Problem Understanding

The program reads the size, array elements and k. It passes the variable-length array to kth_smallest() using its address. The function validates k, sorts the array in ascending order using bubble sort, and returns the element at position k - 1. Duplicate values remain separate while determining the kth position.

2. Standard Test Cases

Test Case	Size (n)	Array Input	k	Expected Output
1	6	7 10 4 3 20 15	3	kth smallest element: 7
2	6	7 10 4 3 20 15	4	kth smallest element: 10
3	6	7 10 4 3 20 15	7	-1
4	5	4 2 2 7 5	3	kth smallest element: 4
5	5	9 -1 6 0 3	1	kth smallest element: -1

3. Algorithm

Main Algorithm

1. Start.
2. Read the size n.
3. Declare a variable-length integer array arr[n].
4. Read n array elements.
5. Read k.
6. Call kth_smallest(arr, n, k) using the array address.
7. Display the returned kth smallest element, or -1 if k is invalid.
8. Stop.

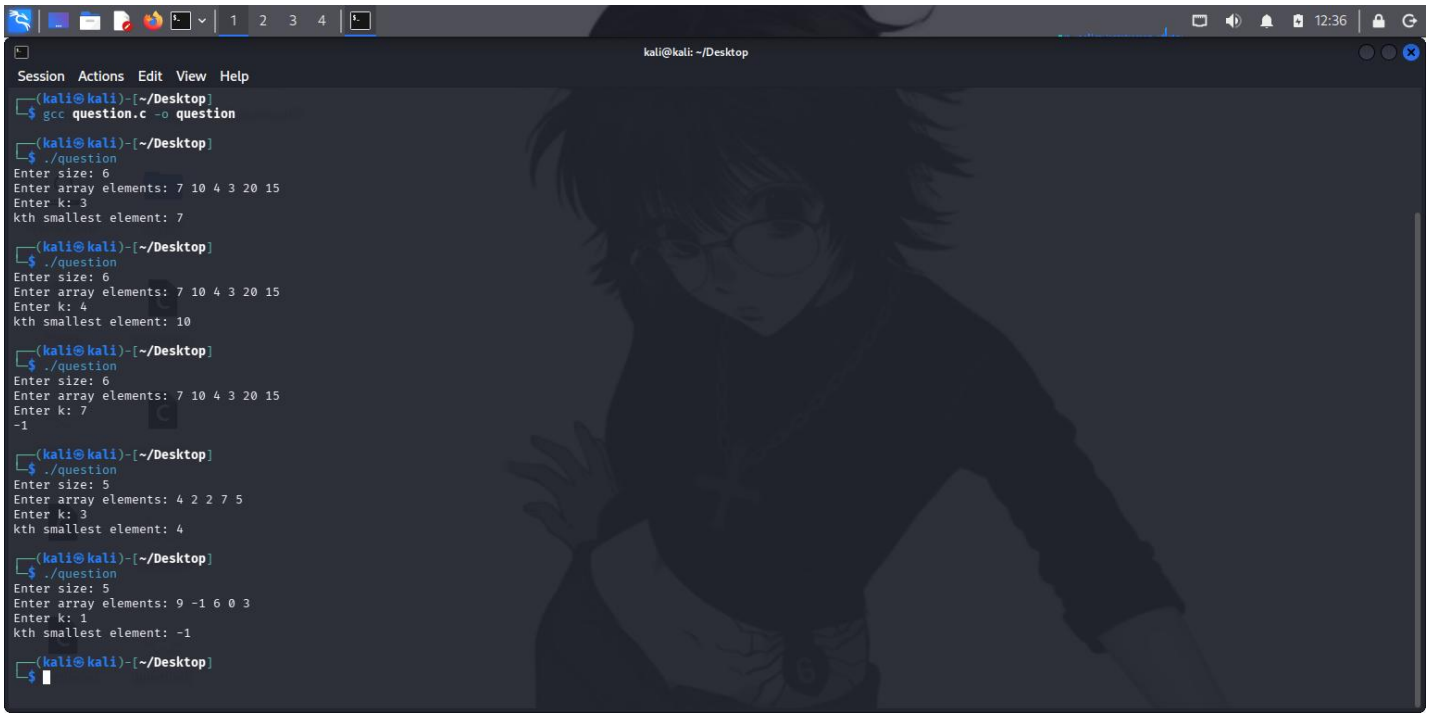
Function: kth_smallest()

1. If k <= 0 or k > n, return -1.
2. Use two nested loops to perform bubble sort on the array.
3. Compare adjacent elements and swap them when the left element is greater than the right element.
4. After sorting, return arr[k - 1].
5. Duplicates are not removed and are counted as separate positions.

4. C Program

```
#include <stdio.h>
int kth_smallest(int *arr, int n, int k)
{
   int i, j, temp;
   if(k <= 0 || k > n)
      return -1;
   for(i = 0; i < n - 1; i++)
   {
      for(j = 0; j < n - i - 1; j++)
      {
         if(*(arr + j) > *(arr + j + 1))
         {
            temp = *(arr + j);
            *(arr + j) = *(arr + j + 1);
            *(arr + j + 1) = temp;
         }
      }
   }
   return *(arr + k - 1);
}
int main()
{
   int n, i, k, result;
   printf("Enter size: ");
   scanf("%d", &n);
   int arr[n];
   printf("Enter array elements: ");
   for(i = 0; i < n; i++)
      scanf("%d", &arr[i]);
   printf("Enter k: ");
   scanf("%d", &k);
   result = kth_smallest(arr, n, k);
   if(k <= 0 || k > n)
      printf("-1\n");
   else
      printf("kth smallest element: %d\n", result);
   return 0;
}
```

5. Compilation and Execution Evidence



6. Verification of All Test Cases

Test Case	Size (n)	Array Input	k	Actual Result	Status
1	6	7 10 4 3 20 15	3	kth smallest element: 7	Pass
2	6	7 10 4 3 20 15	4	kth smallest element: 10	Pass
3	6	7 10 4 3 20 15	7	-1	Pass
4	5	4 2 2 7 5	3	kth smallest element: 4	Pass
5	5	9 -1 6 0 3	1	kth smallest element: -1	Pass